

Guia MIPS64 — Sistemas Computacionais (EI)

Referência completa extraída dos PDFs da disciplina, com as tabelas de instruções, as directivas do assembler, os códigos de entrada/saída e as estruturas de controlo.

Fontes (todas na pasta `Scripts`, prefixo `SC -`):

Ficheiro	Conteúdo
TabelaMIPS64.pdf	Tabela de instruções e directivas (a referência)
Tamanho dos dados.pdf	Bits por tipo de dados em MIPS32 e MIPS64
MIPS64SimulatorIO.pdf	Memory Mapped I/O — o CR e o DR
MIPS - movimentação de dados.pdf	Como mover dados entre registos e memória
Estruturas de controlo-ASM.pdf	<code>if/else</code> , <code>for</code> , <code>while</code> , <code>switch</code> em MIPS
Exercicios MIPS.pdf	Enunciados dos exercícios 1 a 10
Suporte Ex 01-04.pdf	Resolução comentada dos exercícios 1 a 4
Bases Numericas.pdf	Bases 2, 8, 10, 16 e conversões
MIPS.pdf	Aulas teóricas — registos, interface, case sensitivity
Winmips Tutorial.pdf	Manual do simulador (em inglês)

1. Registos

1.1 Registos inteiros (GPR)

- **32 registos** de uso geral, `r0` a `r31`, de **64 bits** cada
- **`r0` é sempre 0.** Não se pode escrever nele. Serve para:
 - descartar um resultado (`dadd r1, r2, r0` deixa `r2` em `r1`)
 - fornecer o valor zero quando é preciso um
 - ser a base 0 nos endereços — `A(r0)` é o mesmo que `A`
- **`r31`** é usado implicitamente por `jal` (guarda o endereço de retorno). Pode também ser usado como registo normal.

1.2 Registos de vírgula flutuante (FPR)

- **32 registos** `f0` a `f31`
- 64 bits se o bit **FR** do registo `CP0 Status` for 1; 32 bits se for 0
- `f0` e `f1` juntos formam o par `f0:f1` de 64 bits

1.3 Outros

- **PC** (Program Counter) — endereço da instrução seguinte

2. Maiúsculas e minúsculas

Esta é uma das armadilhas mais fáceis de tropeçar.

Case sensitive (diferencia maiúsculas)	Case insensitive (não diferencia)
Directivas: <code>.word</code> , <code>.data</code> , <code>.text</code>	Instruções: <code>dadd</code> = <code>DADD</code>
Nomes de variáveis: <code>A</code> ≠ <code>a</code>	Registos: <code>r1</code> = <code>R1</code>
Etiquetas: <code>IMPAR</code> ≠ <code>impar</code>	Hexadecimal: <code>0xba7</code> = <code>0xBA7</code>
Pseudo-nomes: <code>\$ra</code> ≠ <code>\$RA</code>	

O caso engana: `r1` e `R1` são o mesmo registo, mas `$ra` e `$RA` são pseudo-nomes **diferentes**.

3. Tamanho dos dados

3.1 Inteiros

Elemento	MIPS32	MIPS64
R1 (registro)	32 bits	64 bits
.word	32	64
.word32	—	32
.word16	—	16
.half	16	—
.byte	8	8
LD	—	64
LW	32	32
LH	16	16
LB	8	8

3.2 Stores

Elemento	MIPS32	MIPS64
SD	—	64
SW	32	32
SH	16	16
SB	8	8

3.3 Vírgula flutuante

Elemento	MIPS32	MIPS64
F1 (registro)	32	64
D0 (F0:F1)	64	—
.float	32	32 *
.double	64	64
L.D	64	64
L.S	32	32

* `.float` não é implementado no WinMIPS64. Usar `.double`.

4. Tabela de instruções

Formato: instrução destino, origem1, origem2 → o que faz.

4.1 Aritmética

Instrução	Exemplo	Significado
Add	dadd R1, R2, R3	R1 = R2 + R3
Add Immediate	daddi R1, R2, #100	R1 = R2 + 100
Subtract	dsub R1, R2, R3	R1 = R2 - R3
Multiply (signed)	dmul R1, R2, R3	R1 = R2 * R3
Multiply (unsigned)	dmulu R1, R2, R3	R1 = R2 * R3
Divide (signed)	ddiv R1, R2, R3	R1 = R2 / R3
Divide (unsigned)	ddivu R1, R2, R3	R1 = R2 / R3
Add unsigned	daddu R1, R2, R3	R1 = R2 + R3
Add immediate unsigned	daddui R1, R2, #imm	R1 = R2 + imm
Subtract unsigned	dsubu R1, R2, R3	R1 = R2 - R3

4.2 Lógica

Instrução	Exemplo	Significado
And	and R1, R2, R3	$R1 = R2 \text{ AND } R3$
And Immediate	andi R1, R2, #20	$R1 = R2 \text{ AND } 20$
Or	or R1, R2, R3	$R1 = R2 \text{ OR } R3$
Or Immediate	ori R1, R2, #imm	$R1 = R2 \text{ OR } \text{imm}$
Xor	xor R1, R2, R3	$R1 = R2 \text{ XOR } R3$
Xor Immediate	xori R1, R2, #imm	$R1 = R2 \text{ XOR } \text{imm}$
Load upper immediate	lui R1, imm	carrega a metade alta do registro

4.3 Deslocamentos (shifts)

Instrução	Exemplo	Significado
Shift left logical	dsll R1, R2, #3	$R1 = R2 \ll 3 (= R2 * 2^3)$
Shift right logical	dsrl R1, R2, #4	$R1 = R2 \gg 4 (= R2 / 2^4)$
Shift right arithmetic	dsra R1, R2, #4	como o anterior, mas preserva o sinal
Shift left by variable	dsllv R1, R2, R3	$R1 = R2 \ll R3 (= R2 * 2^{R3})$
Shift right by variable	dsrlv R1, R2, R3	$R1 = R2 \gg R3$
Shift right arith. by variable	dsrav R1, R2, R3	$R1 = R2 \gg R3$ (com sinal)

O deslocamento imediato vai até **31** (5 bits). Um shift maior é ilegal.

4.4 Comparação (set — não salta, escreve)

Instrução	Exemplo	Significado
Set less than	slt R1, R2, R3	Se $R2 < R3$ então $R1 = 1$, senão $R1 = 0$
Set less than imm.	slti R1, R2, #20	Se $R2 < 20$ então $R1 = 1$, senão $R1 = 0$
Set less than unsigned	sltu R1, R2, R3	idem, sem sinal
Set less than imm. unsigned	sltiu R1, R2, #imm	idem, sem sinal

4.5 Movimentação de dados

Instrução	Formato	Tamanho
Load	ld reg, imm(reg)	64 bits (doubleword)
Load word	lw reg, imm(reg)	32 bits
Load word unsigned	lwu reg, imm(reg)	32 bits, sem sinal
Load half-word	lh reg, imm(reg)	16 bits
Load half-word unsigned	lhu reg, imm(reg)	16 bits, sem sinal
Load byte	lb reg, imm(reg)	8 bits
Load byte unsigned	lbu reg, imm(reg)	8 bits, sem sinal
Store	sd reg, imm(reg)	64 bits
Store word	sw reg, imm(reg)	32 bits
Store half-word	sh reg, imm(reg)	16 bits
Store byte	sb reg, imm(reg)	8 bits

O endereço é sempre **base + deslocamento**:

Endereço	Significado
A(R0)	A — porque r0 é 0
A(R2)	A + R2
(R2)	0 + R2
200(R2)	200 + R2

4.6 Vírgula flutuante

Instrução	Exemplo	Significado
Load FP	l.d F1, 100(R1)	F1 = memória[R1 + 100]
Store FP	s.d F2, 100(R1)	memória[R1 + 100] = F2 (64 bits)
Add	add.d F1, F2, F3	F1 = F2 + F3 (dupla)
Subtract dupla	sub.d F1, F2, F3	F1 = F2 - F3 (dupla)
Subtract simples	sub.s F1, F2, F3	F1 = F2 - F3 (simples, 32 bits)
Multiply dupla	mul.d F1, F2, F3	F1 = F2 * F3 (dupla)
Divide simples	div.s F1, F2, F3	F1 = F2 / F3 (simples)
Move	mov.d F1, F2	F1 = F2
Conversão int → double	cvt.d.l F1, F1	inteiro de 64 bits para double
Conversão double → int	cvt.l.d F1, F1	double para inteiro de 64 bits
Mover int → FP	mtc1 R1, F1	F1 = R1
Mover FP → int	mfc1 R1, F1	R1 = F1
Comparar (seta flag)	c.lt.d, c.le.d, c.eq.d	põe a flag se a comparação for verdadeira
Branch na flag FP	bc1f imm, bc1t imm	salta se a flag for falsa / verdadeira

4.7 Saltos condicionais

Instrução	Exemplo	Salta se...
Branch equal zero	beqz R1, etiqueta	R1 = 0 (o mesmo que beq R1, R0, etiqueta)
Branch not equal zero	bnez R1, etiqueta	R1 != 0 (o mesmo que bne R1, R0, etiqueta)
Branch equal	beq R1, R2, etiqueta	R1 = R2
Branch not equal	bne R1, R2, etiqueta	R1 != R2

4.8 Saltos incondicionais

Instrução	Exemplo	Significado
Jump	j etiqueta	salta para a etiqueta
Jump register	jr R31	salta para a instrução no endereço guardado em R31
Jump and link	jal etiqueta	salta e guarda em R31 o endereço de retorno (PC + 4)

4.9 Miscelâneas

Instrução	Significado
halt	para a execução do programa
nop	não faz nada
movz R1, R2, R3	move se R3 = 0
movn R1, R2, R3	move se R3 != 0

5. Directivas do assembler

Directiva	Significado
<code>.data</code>	início do segmento de dados
<code>.text</code>	início do segmento de código
<code>.code</code>	o mesmo que <code>.text</code>
<code>.org <n></code>	endereço inicial
<code>.space <n></code>	reserva <code>n</code> bytes vazios na memória
<code>.asciiz <s></code>	cadeia terminada por nulo (<code>\0</code>)
<code>.ascii <s></code>	cadeia sem nulo
<code>.align <n></code>	alinha o próximo endereço a uma fronteira de <code>n</code> bytes
<code>.word <n1>,<n2></code>	palavras de 64 bits
<code>.word32 <n1>,<n2></code>	palavras de 32 bits
<code>.word16 <n1>,<n2></code>	palavras de 16 bits
<code>.byte <n1>,<n2></code>	sequência de bytes (8 bits)
<code>.double <f1>,<f2></code>	vírgula flutuante de 64 bits

`<n>` é um número, `<s>` uma cadeia entre aspas, `<n1>,<n2>` inteiros separados por vírgula, `<f1>,<f2>` reais separados por vírgula.

.asciiz vs .ascii: o simulador precisa do zero no fim para saber onde a cadeia acaba. Sem ele, continua a ler memória a seguir e mostra lixo. O `ex_test5.s` usa `.ascii + .byte 13, 0` para ter um `\r` **antes** do zero.

O imediato é limitado a 16 bits. Um deslocamento imediato é limitado a 5 bits (shift maior que 31 é ilegal).

6. Memory Mapped I/O — o CR e o DR

O simulador não tem chamadas ao sistema: não existe `print` nem `scanf`. Em vez disso há **dois registos de hardware** que se escrevem como se fossem memória normal:

```
CR: .word32 0x10000    ; CONTROL — o gatilho
DR: .word32 0x10008    ; DATA   — a carga
```

6.1 A regra

Sempre a mesma, em duas partes:

1. Escreve-se o valor (ou o endereço) em **DR**
2. Escreve-se em **CR** um código que diz **o que fazer** com o que está em DR

É o passo 2 que dispara a acção. Escrever no DR, sozinho, não produz nada visível. E a ordem conta: DR primeiro, CR depois. Inverter faz o CR disparar com o que estivesse no DR de uma operação anterior.

Os endereços carregam-se com `lwu` (unsigned), porque `0x10000` e `0x10008` lidos como 32 bits **com sinal** seriam negativos e o endereço daria a volta:

```
lwu r1, CR(r0)    ; r1 = 0x10000
lwu r2, DR(r0)    ; r2 = 0x10008
```

6.2 Os 9 códigos

CR	Operação	O que vai no DR	Direcção
1	Mostrar inteiro sem sinal	o inteiro	saída
2	Mostrar inteiro com sinal	o inteiro	saída
3	Mostrar vírgula flutuante	o real	saída
4	Mostrar cadeia	o endereço da cadeia	saída
5	Desenhar um pixel	DATA+5 = x, DATA+4 = y, DATA = cor RGB	saída
6	Limpar o ecrã de texto	nada	saída
7	Limpar o ecrã gráfico	nada	saída
8	Ler do teclado	o valor lido	entrada
9	Ler um byte, sem eco	o byte	entrada

O código 4 é o que trapaceia: no DR não vai o texto, vai o **endereço** do texto. É a diferença entre o *que mostrar* e *onde está o que mostrar*. Nos códigos 1, 2 e 3 vai o valor em si.

O código 8 inverte a ordem: é o próprio CR que escreve no DR, por isso não há `sd` a preparar nada antes. É `sd código,(r1)` seguido de `ld valor,(r2)`.

6.3 Código 1 — mostrar inteiro sem sinal

```
sd r4, (r2)      ; DR = o valor a mostrar
daddi r3, r0, 1  ; r3 = 1
sd r3, (r1)      ; CR = 1 -> mostra
```

6.4 Código 2 — mostrar inteiro com sinal

```
sd r4, (r2)      ; DR = o valor (pode ser negativo)
daddi r3, r0, 2  ; r3 = 2
sd r3, (r1)      ; CR = 2 -> mostra
```

Quando usar 1 e quando usar 2: se o valor puder ser negativo, tem de ser o 2. Com o 1 um valor negativo aparece como um número enorme.

6.5 Código 4 — mostrar cadeia

```
Mensagem: .asciiz "Arquitetura"

daddi r4, r0, Mensagem ; r4 = ENDEREÇO da cadeia
sd r4, (r2)            ; DR = esse endereço
daddi r3, r0, 4         ; r3 = 4
sd r3, (r1)            ; CR = 4 -> mostra as letras
```

As letras continuam paradas na `.data`; o que viaja no DR é só o endereço.

6.6 Código 8 — ler do teclado

```
daddi r3, r0, 8      ; r3 = 8
sd r3, (r1)          ; CR = 8 -> o cursor fica a piscar
ld r4, (r2)          ; r4 = o valor digitado
```

6.7 Código 6 — limpar o ecrã

```
daddi r3, r0, 6
sd r3, (r1)          ; CR = 6 -> limpa
```

7. Movimentação (cópia) de dados

7.1 Notação

Símbolo	Significa
R1	registo
M	memória
Const	constante pequena (valor absoluto $< 2^{15}$)
ConstP	constante maior, já em memória
a	variável em memória

7.2 Inteiros

De → Para	Instrução	Notas
R1 → R2 (cópia)	daddi r2, r1, 0	a forma de copiar um registo
R1 → R2 (cópia)	dadd r2, r1, r0	idem, r0 é 0
R1 → M	sd r1, a(r0)	guarda
M → R2	ld r2, a(r0)	carrega
→ R2 (const pequena)	daddi r2, r0, 1000	soma r0 com a constante
→ R2 (const grande)	ld r2, ConstP(r0)	tem de estar em memória

Constante grande, por exemplo:

```
Const: .word 39000
ld r2, Const(r0)
```

7.3 Vírgula flutuante

De → Para	Instrução
F1 → F2	mov.d f2, f1
F1 → M	s.d f1, a(r0)
M → F2	l.d f2, ConstF(r0)
→ F2 (const)	l.d f2, ConstF(r0) com ConstF: .double 3.14

7.4 Inteiro ↔ vírgula flutuante

De → Para	Instrução
R1 → F1	mtc1 r1, f1
F1 → R1	mfc1 r1, f1
R1 (inteiro) → F1 (2.0)	mtc1 r1, f1 + cvt.d.l f1, f1
F1 (2.0) → R1 (2)	cvt.l.d f1, f1 + mfc1 r1, f1

8. Estruturas de controlo em MIPS

Em MIPS **não existe** instrução de `if` com bloco. Só há saltos condicionais, e o `if/else` é construído com eles.

8.1 Condições simples

Caso	Estrutura	MIPS
1	Se <code>r1 = r2</code>	bne r1, r2, NaoFaz op1 NaoFaz:
1a	Se <code>r1 != r2</code>	beq r1, r2, NaoFaz op1 NaoFaz:
2	Se <code>r1 >= r2</code>	slt r3, r1, r2 bnez r3, NaoFaz op1 NaoFaz:
3	Se <code>r1 <= r2</code>	slt r3, r2, r1 bnez r3, NaoFaz op1 NaoFaz:
4	Se <code>r1 < r2</code>	slt r3, r1, r2 beqz r3, NaoFaz op1 NaoFaz:
4a	Se <code>r1 > r2</code>	slt r3, r2, r1 beqz r3, NaoFaz op1 NaoFaz:

O truque de `>=` e `<=`: não existem. Inverte-se o `slt` e inverte-se o salto. Com os dois registos trocados, o `<` resolve o `<=`.

8.2 if / else

Solução A — saltar para o `then`:

```

beq r1, r2, Se
op2           ; o "else"
j FimSe
Se:
op1           ; o "then"
FimSe:

```

Solução B — saltar para o `else`:

```

bne r1, r2, Else
op1
j FimSe
Else:
op2
FimSe:

```

8.3 Condições compostas

OU — “se `r1 = r2` ou `r3 = r4`”:

```

beq r1, r2, Se
beq r3, r4, Se
op2           ; falhou a primeira E a segunda -> op2
j FimSe
Se:
op1
FimSe:

```

E — “se `r1 = r2` e `r3 = r4`”:

```

bne r1, r2, Else
bne r3, r4, Else
op1
j FimSe
Else:
op2
FimSe:

```

8.4 for

```

daddi r5, r0, 0      ; i = 0
For:
slt r1, r5, r6      ; i < limite ?
beqz r1, FimFor     ; se nao, sai
op1
daddi r5, r5, 1      ; i = i + 1
j For               ; volta ao topo
FimFor:

```

8.5 while

```

While:
beq r1, r2, FimWhile ; sai quando forem iguais
op1
j While
FimWhile:

```

8.6 do ... while

```

Faz:
op1
bne r1, r2, Faz      ; repete enquanto forem diferentes

```

8.7 switch

```

daddi r2, r0, -3
bne r1, r2, Testa0
op1
j FimSwitch

Testa0:
bnez r1, Testa43
op2
j FimSwitch

Testa43:
daddi r2, r0, 43
bne r1, r2, Default
op3
j FimSwitch

Default:
op4
FimSwitch:

```

Solução alternativa (comparar contra a constante com `beq`):

```

daddi r2, r0, -3
beq r1, r2, FazOp1
beq r1, r0, FazOp2
daddi r2, r0, 43
beq r1, r2, FazOp3
op4
j FimSwitch

```

```

FazOp1: op1
j FimSwitch
FazOp2: op2
j FimSwitch
FazOp3: op3
FimSwitch:

```

9. Bases numéricas

9.1 Correspondência

Decimal	Binário	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

9.2 Regras de conversão

De → Para	Método
Binário → Decimal	multiplicar cada dígito pelo valor da potência de 2 correspondente e somar
Decimal → Binário	divisões sucessivas por 2 até o quociente dar 0
Binário → Octal	grupos de 3 dígitos, da direita para a esquerda
Binário → Hex	grupos de 4 dígitos, da direita para a esquerda
Octal → Binário	cada símbolo vira 3 dígitos
Hex → Binário	cada símbolo vira 4 dígitos
Decimal → Octal/Hex/n	divisões sucessivas pela base até o quociente dar 0

Se o último grupo à esquerda não tiver os dígitos todos, completa-se com zeros à esquerda — e esses zeros podem ser omitidos no resultado.

Exemplo: 110101 (base 2) = $32 + 16 + 0 + 4 + 0 + 1 = 53$ (base 10)

9.3 Números negativos

Há várias representações. A aula apresenta o **sinal-módulo**: acrescenta-se um bit de sinal à esquerda — 0 positivo, 1 negativo.

Em 8 bits:

Decimal	Binário	8 bits
23	10111	00010111
-15	1111	10001111
11	1011	00001011
-9	1001	10001001

Atenção: o MIPS **não** usa sinal-módulo. Usa complemento a dois, e é por isso que 1 e 2 na tabela do CR dão resultados diferentes para números negativos.

10. Simulador WinMIPS64

10.1 As sete janelas

Janela	O que mostra
Pipeline	esquema das 5 etapas e em que etapa está cada instrução
Code	endereço (4 hex), código máquina (8 hex), código assembler
Data	memória de dados, em blocos de 64 bits
Registers	valores dos registos
Statistics	ciclos, instruções, CPI, stalls
Cycles	diagrama temporal do pipeline
Terminal	terminal de I/O, com alguma capacidade gráfica

Cada posição de memória de dados ocupa 64 bits (16 hex), por isso o endereço incrementa de 8 em 8. Cada instrução ocupa 4 B.

10.2 Cores no pipeline

As cores indicam a etapa, e a cor de uma instrução é a etapa em que ela **completa no próximo ciclo**:

Cor	Etapa
Amarelo	IF (Instruction Fetch)
Azul	ID (Instruction Decode)
Vermelho	EX (Execute)
Verde	MEM (Memory)
Roxo	WB (Write Back)

No Clock Cycle Diagram, uma instrução que causa stall fica a **azul**, e as que ficam presas atrás ficam a **cinzento**.

10.3 Teclas

Tecla	Ação
F4	executar tudo de uma vez (Execute → Run to)
F7	executar um ciclo de cada vez (Execute → Single Cycle)
F10	reiniciar a simulação (Reload)
Ctrl+O	File → Open
Ctrl+R	File → Reset MIPS64

10.4 Reset, Reload e Full Reset

Opção	Efeito
Reset MIPS64 (Ctrl+R)	volta ao início do código, não apaga a memória
Reload (F10)	volta ao início do código e apaga a memória
Full Reset	reinicia tudo; é preciso voltar a carregar o programa

O tutorial em inglês descreve o **Full Reset** como o que apaga a memória de dados, e o **Reload** como “reiniciar a simulação”. A versão portuguesa (aula teórica) é a que está na tabela acima.

10.5 Configuração

Em `Configure → Architecture` mudam-se a estrutura da pipeline de vírgula flutuante e o tamanho das memórias.

Em `Configure`: **Multi-Step**, **Enable Forwarding** (deve estar ligado), **Enable Branch Target Buffer** e **Enable Delay Slot**.

Forwarding ligado ou desligado muda o resultado. Sem forwarding, as instruções que dependem de um registo ficam à espera em ID até a anterior terminar em WB. Com forwarding, o valor passa adiante mais cedo — e é por isso que no exemplo do tutorial a soma só custa um stall em vez de vários.

11. Exercícios e ficheiros correspondentes

Exercício	Enunciado	Ficheiro local
1a	<code>c = a + b</code> com <code>a=10</code> , <code>b=8</code>	<code>ex1A.s</code>
1b	<code>c = a + b + z</code>	<code>ex1A.s</code>
2a/2b	Trocar duas variáveis em memória	<code>ex2A.s</code> — tinha o bug da versão 2b; ver §12.1
3	Enviar o resultado para o terminal	<code>ex3A.s</code>
4	Obter os valores do utilizador	<code>ex4A.s</code>
5a	Mostrar “Hello World!”	<code>ex5A.s</code>
5b	Se <code>a</code> e <code>b</code> forem iguais, “Iguais!”	<code>ex_test3.s</code>
5c	Caso contrário, “O maior é:” + o maior	<code>ex_test3.s</code>
—	(extra) par ou ímpar	<code>ex_test4.s</code>
—	(extra) subtração de dois valores	<code>ex_test2.s</code>
—	(extra) contar de 1 a <code>n</code> com validação	<code>ex_test5.s</code>
—	(extra) soma mínima com saída	<code>exteste.s</code>

Os exercícios **6 a 10** não têm ficheiro local:

Exercício	O que pede
6	Número secreto; o programa diz se é maior ou menor; mensagens de parabéns por nº de tentativas
7	Binário com a rotina <code>ex07_Rotina_Bin.s</code> : portas (bits de um byte); endereço IPv4 com máscara /25
8	Vectores de 10 inteiros de 64 bits, mostrar e somar 1 ao 4º; depois a mesma coisa em 16 bits
9	Contar caracteres e espaços; minúscula → maiúscula; converter uma cadeia para inteiro
10	Rotina <code>Max</code> com parâmetros e resultado pela pilha; usar <code>\$s0</code> com guarda/restauração; chamar de outra rotina

12. Bugs e erros encontrados

12.1 `ex2A.s` não trocava nada — bug encontrado e corrigido

Já foi corrigido. Fica aqui o registo porque é o erro mais discreto de todos e vale a pena saber reconhecer.

Estado inicial: A = 10, B = 8. O código original era:

```
ld r4, A(r0)    ; r4 = 10
ld r5, B(r0)    ; r5 = 8
dadd r3, r4, r0 ; r3 = 10
dadd r4, r5, r0 ; r4 = 8    <- r4 passa a ter o valor de B
dadd r5, r3, r0 ; r5 = 10   <- r5 passa a ter o valor de A
sd r4, B(r0)    ; B = 8    (B já era 8 -- não muda nada)
sd r5, A(r0)    ; A = 10   (A já era 10 -- não muda nada)
```

No fim: A = 10, B = 8. **Isto não é uma troca.** O 10 nunca foi para o B, e o 8 nunca foi para o A.

A causa: o ficheiro misturava a ordem de `store` do exercício **2a** com a troca de registos do exercício **2b**. Depois das três `dadd`, o `r4` já tem o valor de B e o `r5` o valor de A — por isso têm de ser escritos nos sítios **trocados**. A correcção são as duas últimas linhas:

```
sd r4, A(r0)    ; A = 8    (r4 tem o valor que estava em B)
sd r5, B(r0)    ; B = 10   (r5 tem o valor que estava em A)
```

Conferir as três versões:

Versão	Troca de registos	Stores	Resultado
Ex 2a (PDF)	nenhuma	sd r4,B / sd r5,A	troca correcta
Ex 2b (PDF)	3 dadd	sd r4,A / sd r5,B	troca correcta
ex2A.s antes	3 dadd	sd r4,B / sd r5,A	não fazia nada
ex2A.s agora	3 dadd	sd r4,A / sd r5,B	troca correcta

A regra que daqui sai: nas duas soluções o store é `r4` para uma variável e `r5` para a outra — o que muda é **qual**. O 2a não mexe nos registos, por isso `r4` ainda tem o valor de A e vai para B. O 2b troca os registos, por isso `r4` passa a ter o valor de B e vai para A. Guardar a ordem do 2a depois das `dadd` do 2b dá um programa que corre e não faz nada.

Como este bug escapou: o programa monta, executa sem erros, e não dá nenhum aviso. Só se vê olhando para os valores de A e B no fim. Um teste que confirme “o assembler não deu erro” não o apanha — é preciso verificar o *resultado*, que é o que o exercício pede.

12.2 Erros nos PDFs

Onde	O que diz	O que é
TabelaMIPS64.pdf	dsll R1, R2, #3 → R1 = R2 << 2	o exemplo e a fórmula não batem: tem de ser << 3
TabelaMIPS64.pdf	cvt.w.d → “convert 32-bit integer to floating-point”	está ao contrário: cvt.w.d converte de double para inteiro de 32 bits
TabelaMIPS64.pdf	xori aparece duas vezes na lista	duplicado da linha anterior
Bases Numericas.pdf	tabela de 4 bases, linha do 8: binário 1001	é 1000 — 1001 é 9 e já aparece na linha seguinte
Suporte Ex 01-04.pdf	na solução do ex4a ficam <code>ld r4,A(r0)</code> e <code>ld r5,B(r0)</code> depois das leituras do teclado	código morto que sobrescreve os valores lidos com 10 e 8. O <code>ex4A.s</code> local acertou ao removê-los
Tamanho dos dados.pdf	.word em MIPS32 marcado como 32 bits	em MIPS64 .word são 64 bits; o .word32 é que é de 32

12.3 Diferenças entre documentos

Assunto	Uma fonte	Outra fonte
Ficheiro do exercício 1	ex1a-sum.s (Moodle)	ex1A.s (local)
Tabela de instruções	SC-TabelaMIPS64 v1.pdf	SC - TabelaMIPS64.pdf (actual)
Directivas	.text	.code — as duas servem, a aula usa ambas

13. Receita rápida — os quatro blocos mais usados

13.1 Carregar um valor da memória para um registo

```
ld r4, A(r0)
```

13.2 Guardar um registo na memória

```
sd r3, C(r0)
```

13.3 Colocar uma constante pequena num registo

```
daddi r2, r0, 1000
```

13.4 Colocar uma constante grande num registo

```
Const: .word 39000
ld r2, Const(r0)
```

14. Bibliografia

- Patterson & Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5ª ed., 2013
- Hennessy & Patterson, *Computer Architecture: A Quantitative Approach*, 6ª ed., 2017
- WinMIPS64 — <http://indigo.ie/~mscott/>

Créditos dos materiais: Nuno Veiga, Rui Vasco Monteiro, Rolando Miragaia.